TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 1

Why Kubernetes and Architecture

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 1: Why Kubernetes? Architecture and how it works inside 3

1.1 The problem: containers without a manager 3

1.2 What is container orchestration? 4

1.3 Advantages and disadvantages 4

1.4 Kubernetes architecture 5

1.5 Desired state and the control loop 6

1.6 What happens when you run kubectl apply 7

1.7 Live lab: your first cluster 8

Class 1 - Top 20 interview questions 10

K8S · CLASS 01 · 60-90 minutes

Why Kubernetes? Architecture and how it works inside

By the end you can explain why companies use Kubernetes, name every component of a cluster, and run your own cluster on your laptop.

What you will learn today

- The problem with running containers by hand
- What container orchestration means, and where Kubernetes came from
- Advantages and disadvantages of Kubernetes
- Control plane and worker node components - what each one does
- Desired state and the reconciliation (control) loop
- What happens step by step when you run `kubectl apply`
- Lab: start minikube, explore the cluster, watch self-healing

Part	Topic	Time
1	Story and the problem with plain Docker	10 min
2	Orchestration, history, pros and cons	15 min
3	Architecture and the apply flow	25 min
4	Live lab	25 min
5	Recap and interview questions	10 min

1.1 The problem: containers without a manager

Ravi's story

QuickBite runs 12 containers on 3 servers using `docker run`. On Friday night the orders container ran out of memory and died. Docker did not move it anywhere, nobody was paged, and customers saw "Something went wrong" for 40 minutes. Then traffic tripled and Ravi had to SSH into each server and start more containers by hand. On Monday a new release needed to go out - he stopped the old containers and started new ones, causing 3 minutes of downtime.

Plain Docker is excellent at running a container on **one** machine. As soon as you have many machines and many containers, you need answers to these questions:

Question	With plain Docker	With Kubernetes
A container crashes at 2 AM. Who restarts it?	You, after you wake up	Kubernetes, within seconds
A whole server dies. What happens to its containers?	They are gone until you recreate them	They are recreated on healthy servers
Traffic triples. How do we add copies?	SSH and docker run, one by one	`kubectl scale` or automatic autoscaling

Question	With plain Docker	With Kubernetes
Which server has free CPU for a new container?	You check manually	The scheduler decides
How do containers find each other when IPs change?	Hard-coded IPs, links	Services with stable DNS names
How do we release a new version?	Stop old, start new (downtime)	Rolling update, zero downtime
Where do passwords and config live?	Env files on each server	ConfigMaps and Secrets

1.2 What is container orchestration?

Orchestration means automatically deploying, scaling, networking, healing and updating containers across a group of machines. You describe *what* you want ("3 copies of the orders app, version 2.1, reachable on port 80") and the orchestrator figures out *how* to make it happen and keeps it that way.

> **Everyday example**
> Think of a large restaurant chain. The **head office** does not cook. It decides how many cooks each branch needs, replaces a cook who calls in sick, opens extra counters on festival days, and rolls out the new menu branch by branch so no branch is ever closed. Kubernetes is the head office; your servers are the branches; containers are the cooks.

The name **Kubernetes** is Greek for *helmsman* - the person who steers a ship. That is why the logo is a ship's wheel. People shorten it to **K8s** (K, then 8 letters, then s).

> **Did you know?**
> Google ran its own internal orchestrator called **Borg** for over a decade, launching billions of containers a week. In 2014 Google open-sourced a fresh system built on those lessons: Kubernetes. In 2015 it was donated to the Cloud Native Computing Foundation (CNCF). Today every major cloud offers it: Amazon EKS, Azure AKS and Google GKE.

Other orchestrators exist - Docker Swarm, HashiCorp Nomad, Amazon ECS - but Kubernetes has become the industry standard, which is why it appears in almost every DevOps job description.

1.3 Advantages and disadvantages

Advantage	What it means in real life
Self-healing	Crashed containers restart; pods on a dead node are recreated elsewhere.
Horizontal scaling	Go from 3 to 30 copies with one command, or automatically on CPU load.
Zero-downtime releases	Rolling updates replace pods gradually; one-command rollback.
Service discovery and load balancing	Apps call each other by name; traffic is spread across copies.
Declarative configuration	Everything is YAML in Git - reviewable, repeatable, auditable.
Efficient use of servers	The scheduler packs containers onto nodes based on CPU and memory.
Portable	The same YAML runs on a laptop, on-premises, on AWS, Azure or GCP.
Huge ecosystem	Helm, Argo CD, Prometheus, Istio and thousands of CNCF tools.

Disadvantage	Why it matters
Steep learning curve	Many new objects and ideas; that is why this course has eleven classes.
Operational complexity	Upgrades, networking, storage and security need skilled people.
Overkill for small apps	A single website on one server does not need a cluster.
Resource overhead	The control plane and agents use CPU and memory themselves.
Cost if misconfigured	Over-sized requests or forgotten load balancers waste cloud money.
Debugging has more layers	A failure could be in the app, pod, node, network or config.

Remember
Use Kubernetes when you have many services, need high availability, scale often, or release frequently. For one small app, plain Docker or a PaaS is simpler.

1.4 Kubernetes architecture

A Kubernetes **cluster** is a group of machines (called **nodes**) split into two roles:

- **Control plane** - the brain. It stores the desired state, makes decisions and exposes the API. In production there are usually 3 control-plane nodes for high availability.
- **Worker nodes** - the muscle. They actually run your containers.

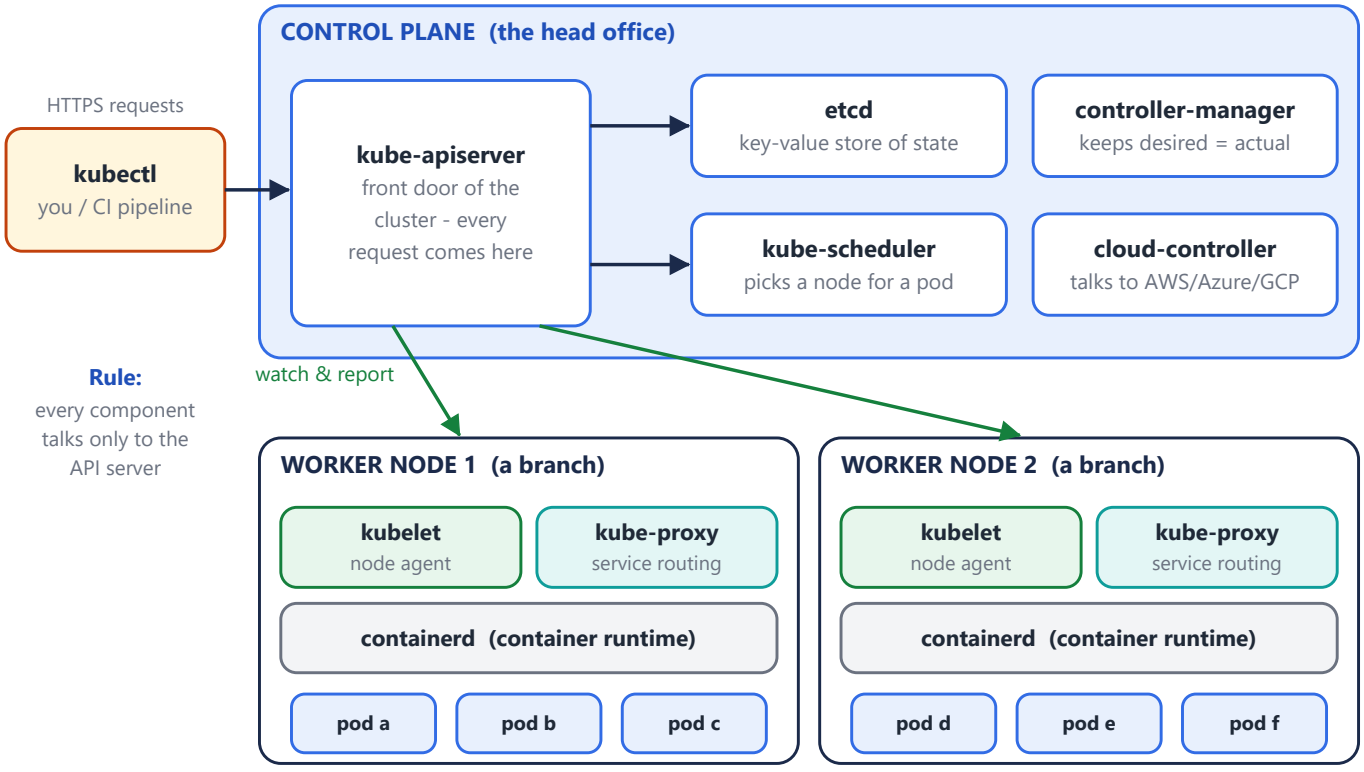


Figure 1.1 - Kubernetes architecture: one control plane managing worker nodes

Control plane components

Component	Job	Everyday example
kube-apiserver	The only entry point. Validates every request (from kubectl, nodes, controllers) and reads/writes etcd.	The reception desk - nobody walks into the office without passing it.
etcd	A distributed key-value database that stores the whole cluster state: every pod, service, secret and config.	The master register book of the head office.
kube-scheduler	Watches for new pods with no node and picks the best node based on resources, rules and affinity.	HR deciding which branch a new cook joins.
kube-controller-manager	Runs many controllers (Deployment, ReplicaSet, Node, Job...) that keep actual state equal to desired state.	Supervisors who keep checking each counter is staffed.
cloud-controller-manager	Talks to the cloud provider: creates load balancers, attaches disks, notices deleted VMs.	The office admin who deals with the landlord.

Worker node components

Component	Job	Everyday example
kubelet	Agent on every node. Receives pod specs from the API server, asks the runtime to start containers, runs health checks, reports status.	The branch manager who reports to head office.
kube-proxy	Programs network rules (iptables/IPVS) so traffic to a Service reaches the right pods.	The telephone operator forwarding calls to the right desk.
Container runtime	Actually runs containers. Usually containerd or CRI-O, talking to the kubelet through the Container Runtime Interface (CRI).	The kitchen equipment the cooks use.
CNI plugin	Gives every pod an IP and connects pods across nodes (Calico, Flannel, Cilium).	The internal phone lines between branches.

Did you know?
Kubernetes stopped using Docker Engine as its runtime ("dockershim" was removed in v1.24). Do not panic - images built with Docker still run perfectly, because they follow the OCI image standard, and containerd runs them directly.

1.5 Desired state and the control loop

This is the single most important idea in Kubernetes. You never say "start a container". You say "I want 3 copies running" - the **desired state**. Controllers then run a never-ending loop: observe what is actually running, compare it with what you asked for, and act to remove the difference. This is called **reconciliation**.

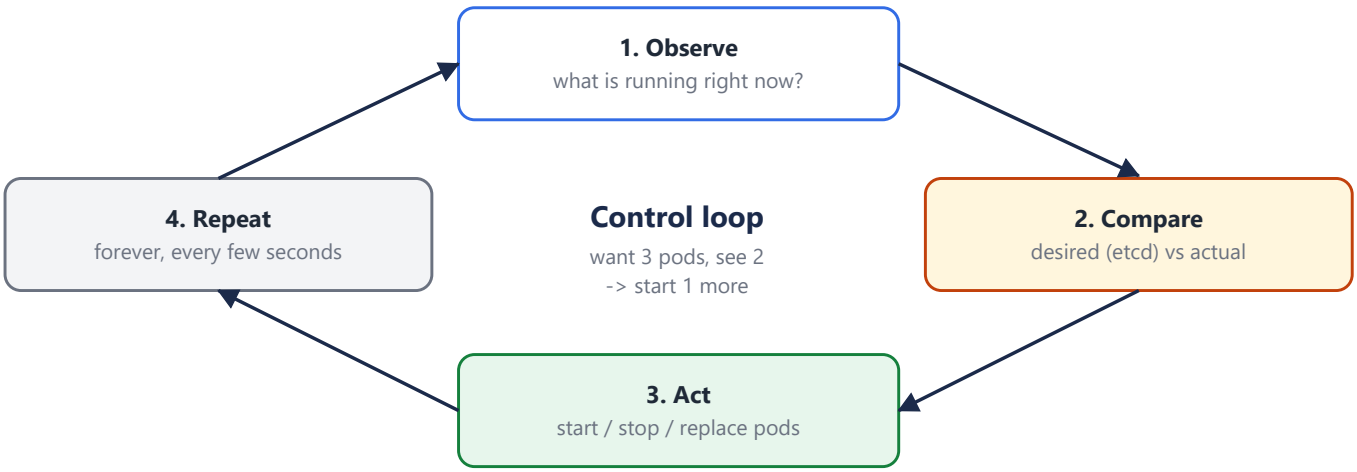


Figure 1.2 - The reconciliation loop that powers self-healing

Everyday example
An air conditioner works the same way. You set 24 degrees (desired state). The thermostat keeps measuring the room (actual state). If it is 27, it cools; if it is 22, it stops. You never tell it "run the compressor for 10 minutes" - you only declare the target.

1.6 What happens when you run kubectl apply

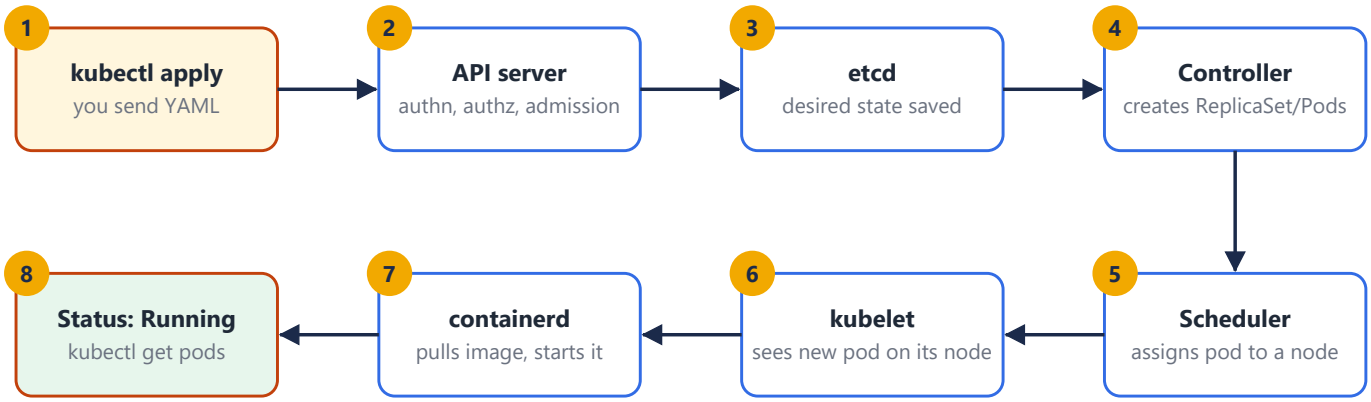


Figure 1.3 - The journey of a Deployment from YAML to running containers

- **1-2.** kubectl sends your YAML to the API server over HTTPS. The API server checks who you are (authentication), whether you are allowed (authorization/RBAC) and runs admission checks.
- **3.** The object is saved in etcd. Nothing is running yet - only the wish is recorded.
- **4.** The Deployment controller notices the new Deployment and creates a ReplicaSet, which creates Pod objects.
- **5.** The scheduler sees pods without a node and writes a node name into each one.
- **6-7.** The kubelet on that node sees a pod assigned to it and tells containerd to pull the image and start the containers.
- **8.** The kubelet reports status back; `kubectl get pods` shows Running.

Remember
Components never talk to each other directly - they all watch the API server. This loose coupling is why Kubernetes is so robust.

1.7 Live lab: your first cluster

Step 1 - Start and inspect the cluster

PowerShell

```
minikube start --driver=docker
kubectl cluster-info
kubectl get nodes -o wide
kubectl get pods -n kube-system
```

In `kube-system` you will see the control-plane components running as pods:

```
NAME                               READY   STATUS    RESTARTS   AGE
coredns-7c65d6cfc9-x2b8l           1/1     Running   0          2m
etcd-minikube                      1/1     Running   0          2m
kube-apiserver-minikube            1/1     Running   0          2m
kube-controller-manager-minikube   1/1     Running   0          2m
kube-proxy-9fk2s                   1/1     Running   0          2m
kube-scheduler-minikube            1/1     Running   0          2m
storage-provisioner                1/1     Running   0          2m
```

Step 2 - Run an app and watch self-healing

PowerShell

```
kubectl create deployment hello --image=nginx:1.27 --replicas=3
kubectl get pods -o wide

# open a second PowerShell window and keep this running:
kubectl get pods -w

# back in the first window, delete one pod:
kubectl delete pod <name-of-any-hello-pod>
```

In the watch window you will see the deleted pod go to **Terminating** and a brand-new pod with a different name appear within seconds. You asked for 3; Kubernetes keeps 3.

Step 3 - Scale and clean up

PowerShell

```
kubectl scale deployment hello --replicas=5
kubectl get pods
kubectl delete deployment hello
minikube dashboard        # opens a web UI in your browser (Ctrl+C to stop)
```

Try it yourself

- Run `kubectl api-resources` to see every object type Kubernetes knows about.
- Run `kubectl get events --sort-by=.metadata.creationTimestamp` after the self-healing demo and read the story the cluster tells.
- Run `minikube ssh` and then `sudo crictl ps` to see the real containers behind your pods.

Common mistake

- Saying "Kubernetes runs Docker containers". It runs **OCI containers** through a CRI runtime such as containerd.
- Thinking the control plane runs your apps. In production, app pods run on worker nodes.
- Forgetting that etcd holds everything - losing etcd without a backup means losing the cluster state.

Class 1 recap

- Kubernetes = an orchestrator that keeps the desired state you declare.
- Control plane: apiserver, etcd, scheduler, controller-manager, cloud-controller-manager.
- Every node: kubelet, kube-proxy, container runtime, CNI.
- Everything goes through the API server; controllers reconcile in a loop.

Class 1 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is Kubernetes?

An open-source container orchestration platform that automates deploying, scaling, healing, networking and updating containerized applications across a cluster of machines. You declare the desired state in YAML and Kubernetes keeps the actual state matching it.

Q2. Why do we need Kubernetes when we already have Docker?

Docker runs containers on a single host. Kubernetes manages containers across many hosts: it restarts failed containers, reschedules them when a node dies, scales them, load-balances traffic, and performs rolling updates. Docker builds and runs images; Kubernetes orchestrates them.

Q3. What is container orchestration?

Automatic management of the container lifecycle across many machines - placement (scheduling), scaling, health monitoring and restarts, networking and service discovery, configuration, and rolling updates.

Q4. Explain the Kubernetes architecture.

A cluster has a control plane and worker nodes. The control plane runs kube-apiserver, etcd, kube-scheduler, kube-controller-manager and optionally cloud-controller-manager. Each worker node runs kubelet, kube-proxy, a container runtime and a CNI plugin. All components communicate through the API server.

Q5. What does the kube-apiserver do?

It is the front end of the control plane and the only component that talks to etcd. It exposes the REST API, authenticates and authorizes requests, runs admission controllers, validates objects, and lets other components watch for changes.

Q6. What is etcd and why is it critical?

etcd is a consistent, distributed key-value store holding the entire cluster state - all objects, configs and secrets. If etcd is lost without a backup the cluster state is lost, so it is run in odd numbers (3 or 5) for quorum and backed up regularly with `etcdctl snapshot save`.

Q7. What does the kube-scheduler do?

It watches for pods with no assigned node and selects a node in two phases: filtering (nodes that have enough resources and satisfy rules like nodeSelector, taints and affinity) and scoring (ranking the remaining nodes). It then binds the pod to the best node. It does not start containers.

Q8. What is the kube-controller-manager?

A single process running many controllers such as the Deployment, ReplicaSet, Node, Job, Endpoints and ServiceAccount controllers. Each controller watches certain objects and reconciles actual state toward desired state.

Q9. What is the role of the kubelet?

The kubelet is the node agent. It watches the API server for pods assigned to its node, asks the container runtime to start them, mounts volumes, runs liveness/readiness/startup probes, and reports pod and node status back.

Q10. What does kube-proxy do?

It runs on each node and maintains network rules (iptables, IPVS or nftables) so that traffic sent to a Service's virtual IP is forwarded to one of the healthy backing pods.

Q11. What is a container runtime? Name some.

Software that actually pulls images and runs containers. The kubelet talks to it through the Container Runtime Interface (CRI). Common runtimes are containerd and CRI-O.

Q12. Did Kubernetes drop Docker? What changed?

Kubernetes removed dockershim in v1.24, so Docker Engine is no longer used directly as the runtime. Images built with Docker still work because they are OCI-compliant; runtimes like containerd run them.

Q13. What is desired state vs actual state?

Desired state is what you declare in manifests (for example 3 replicas of image v2). Actual state is what is really running. Controllers continuously compare the two and act to remove any difference.

Q14. What is a reconciliation loop?

The observe-compare-act cycle that every controller runs forever. It is the reason Kubernetes is self-healing: if a pod disappears, the loop sees 2 instead of 3 and creates a new one.

Q15. What is the difference between imperative and declarative management?

Imperative means giving step-by-step commands such as `kubectl run` or `kubectl scale`. Declarative means writing the desired state in YAML and running `kubectl apply -f`, letting Kubernetes work out the steps. Declarative is preferred in production because files can be versioned in Git.

Q16. What is a node? What are master and worker nodes?

A node is a physical or virtual machine in the cluster. Control-plane (formerly "master") nodes run the management components; worker nodes run application pods.

Q17. What happens if a worker node fails?

The node controller marks it NotReady after missing heartbeats. After a timeout (about 5 minutes by default) its pods are evicted, and controllers such as Deployments recreate them on healthy nodes.

Q18. What happens if the control plane goes down?

Running pods keep running because kubelets and containers work independently, but you cannot deploy, scale, or self-heal until the control plane is back. That is why production uses 3 control-plane nodes.

Q19. What are the advantages and disadvantages of Kubernetes?

Advantages: self-healing, scaling, rolling updates, service discovery, declarative config, portability and a huge ecosystem. Disadvantages: steep learning curve, operational complexity, overhead, and it is overkill for small applications.

Q20. What tools can you use to run Kubernetes locally?

minikube, kind (Kubernetes in Docker), k3d/k3s, and the Kubernetes option in Docker Desktop. For production you would use kubeadm or managed services such as EKS, AKS and GKE.